

Práctica 3: Procesamiento Directo de Archivos BMP mediante Scripts en Python

Lectura de Stream Binario, Umbralización y Binarización de Imágenes

Estudiante: Luis Angel Diaz Dominguez

10 de febrero de 2026

1 Introducción

En la presente práctica desarrollada por **Luis Angel Diaz Dominguez**, se aborda la manipulación de bajo nivel de archivos de imagen en formato BMP (*Bitmap*) utilizando el lenguaje Python en el entorno de Google Colab. El objetivo fundamental consiste en omitir el uso de librerías de alto nivel de procesamiento de imágenes (como OpenCV o PIL) para interactuar directamente con la estructura de bytes del archivo a través de la función `open()` en modo binario ('rb' / 'wb').

2 Metodología y Análisis del Código

El algoritmo de procesamiento lee la cabecera del archivo BMP y salta directamente a la sección de datos de píxeles utilizando un desplazamiento (*offset*) estándar de 54 bytes. Posteriormente, evalúa los valores de color de cada píxel de 24 bits (3 bytes: BGR) y los transforma a blanco o negro según un umbral binario predeterminado.

2.1 Implementación en Python (Google Colab)

A continuación se muestra la estructura del script desarrollado durante la práctica para la binarización de la imagen `volcan.bmp`:

```
# Practica 3 - Procesamiento de Imagenes Binarias
# Estudiante: Luis Angel Diaz Dominguez

blanco = (0xFF, 0xFF, 0xFF)
negro = (0x00, 0x00, 0x00)

file = open('volcan.bmp', 'rb')
fileo = open('example001bin.bmp', 'wb')

# Copiar la cabecera original (54 bytes)
header = file.read(54)
fileo.write(header)

no_pix = 0
limite = (pow(2, 24) - 1) / 2 # Umbral medio para 24 bits

while True:
    pixel_data = file.read(3) # Lectura byte a byte de cada pixel (3 bytes - BGR)
    if len(pixel_data) > 0:
        valor_int = int.from_bytes(bytes(pixel_data), byteorder='little')

        # Clasificacion del pixel segun el umbral
        if valor_int > limite:
```

```

        fileo.write(bytes( blanco ))
    else:
        fileo.write(bytes( negro ))

    no_pix += 1
else:
    break

print('No. de Pixels procesados: ', str(no_pix))
file.close()
fileo.close()

```

3 Resultados y Análisis de Ejecución

Durante la ejecución sobre el archivo de prueba `volcan.bmp`, el programa extrajo de manera secuencial los bloques de 3 bytes representando cada canal de color (Blue, Green, Red).

- **Total de Píxeles Procesados:** 71,743 píxeles.
- **Lectura de Stream Binario:** Se identificó el volcado directo de bytes como `b'\x3d\xed\x1t'`, verificando la codificación *Little-Endian* en la arquitectura del archivo.
- **Salida Generada:** El archivo binario filtrado `example001bin.bmp` se almacenó exitosamente en el directorio de trabajo del entorno virtual.

4 Conclusión

La experiencia práctica permitió a **Luis Angel Diaz Dominguez** comprender la manipulación directa de archivos binarios en memoria secundaria, comprendiendo cómo los algoritmos de cuantización y binarización alteran dinámicamente el contenido espectral de una imagen al segmentar sus tonos mediante umbrales matemáticos.